

Grade de Horários

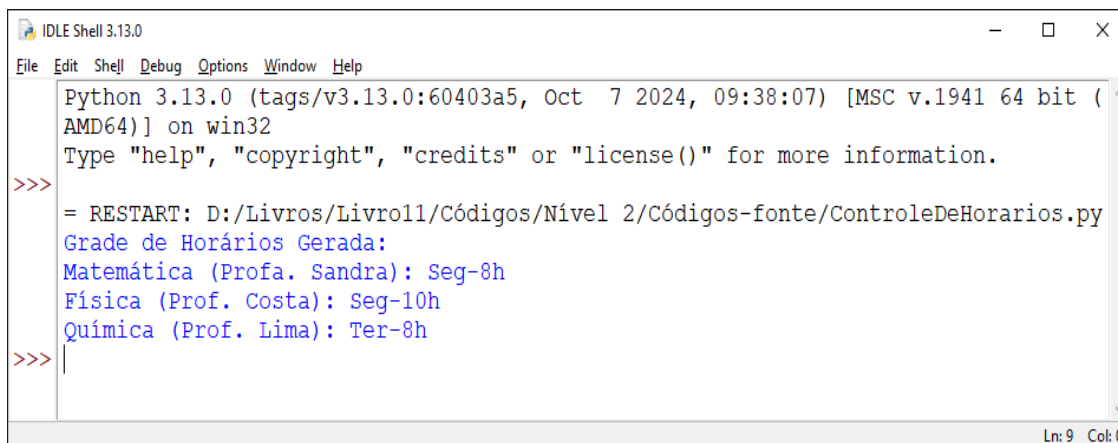
Mário Leite

Criar uma grade de horários é, relativamente, fácil; até com o Excel isto é possível. Mas, mas uma grade de horários (ou escala de trabalho) usando *backtracking* com força bruta, personalizada, que testa combinações válida, que retroceda quando encontra conflitos, que modifica atribuições de cada personagem envolvido e com restrições conforme o cenário, é bem mais complexo.

Pensando nisso, desenvolvi um *script* em Python que gera horários automaticamente usando *backtracking* - uma técnica de tentativa e erro com retrocesso. O programa recebe as listas de disciplinas, horários disponíveis e restrições (como indisponibilidade de professores) e testa combinações válidas até encontrar uma solução completa ou determinar que não é possível. O código apresentado aqui nesta postagem é flexível e pode ser adaptado para escalas de trabalho ou outros cenários com restrições; a estrutura principal do algoritmo e um exemplo de saída.

Conclusão: Na implementação de um gerador de horários baseado em *backtracking* com força bruta, o algoritmo percorre, recursivamente, uma lista de disciplinas (neste caso), testando alocações em horários disponíveis e validando as restrições (conflitos de horário e indisponibilidade de professores). Quando uma alocação inviável é detectada, ocorre o *backtrack* (remoção da última atribuição) até encontrar uma solução completa ou retornar *None*. Com complexidade temporal: $O(N!)$ no pior caso, mas com “podas” via restrições. O código é modular, permitindo facilmente substituições das funções **ObterProfessor()** e **VerificarHorario()** para diferentes domínios (escalas de trabalho, alocação de salas, etc.).

A saída do programa pode ser vista na **figura 1**.



```
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/Livros/Livro11/Códigos/Nível 2/Códigos-fonte/ControleDeHorarios.py
Grade de Horários Gerada:
Matemática (Profa. Sandra): Seg-8h
Física (Prof. Costa): Seg-10h
Química (Prof. Lima): Ter-8h
>>>
```

Figura 1 - Saída do programa “ControleDeHorarios”

```
'''
```

ControleDeHorarios.py

Gera horários (grade de aulas ou escalas de trabalho) usando "*backtracking*", com força bruta que testa combinações válidas e retrocede quando encontra conflitos. Pode ser personalizado, modificando disciplinas, horários, e restrições conforme o cenário.

```
'''
```

```
def GerarHorarios(LstDisciplinas, LstHorarios, LstRestricoes, index=0,
                  LstHorAtuais = None):
    '''
    ***Argumentos:
    LstDisciplinas: Lista de disciplinas/tarefas.
    LstHorarios: Lista de horários disponíveis.
    LstRestricoes: Lista de tuplas (professor, horario):indica indisponibilidades.
    index: Índice da disciplina atual (usado na recursão).
    LstHorAtuais: Lista com alocações parciais como [(disciplina, horario)].

    ***Retorno:
    Lista com alocações válidas ou 'None' se não houver solução.
    '''
    if(LstHorAtuais is None):
        LstHorAtuais = []

    #Caso base: todas as disciplinas foram alocadas
    if(index == len(LstDisciplinas)):
        return LstHorAtuais.copy()

    disciplina = LstDisciplinas[index]
    for horario in LstHorarios:
        #Verifica se o horário é válido para a disciplina atual
        if(VerificarHorario(disciplina, horario, LstHorAtuais, LstRestricoes)):
            LstHorAtuais.append((disciplina, horario))

            #Chamada recursiva para a próxima disciplina
            resultado = GerarHorarios(LstDisciplinas, LstHorarios, LstRestricoes,
                                      index + 1, LstHorAtuais)

            if(resultado is not None):
                return resultado

            #Backtrack: remove a última alocação
            LstHorAtuais.pop()

    return None
```

```
#-----
```

```
def VerificarHorario(disciplina, horario, LstHorAtuais, LstRestricoes):
    """
    Verifica se um horário é válido para uma disciplina, considerando:
    - Conflitos com outros horários (mesmo horário usado).
    - Restrições específicas (ex.: professor indisponível).
    """
    #Extrai o professor da disciplina
    professor = ObterProfessor(disciplina)

    # 1. Verifica se o horário já está ocupado
    for _ h in LstHorAtuais:
        if(h == horario):
            return False

    # 2. Verifica restrições do professor
    if(any((p == professor and h == horario) for p, h in LstRestricoes)):
        return False

    return True
```

```
#-----
```

```

def ObterProfessor(disciplina):
    """
    Exemplo: extrai o professor da disciplina (modifique conforme necessário).
    """
    if("(" in disciplina):
        return disciplina.split("(")[1].strip("(")
    return "SemProfessor"
#=====
#Programa principal
if(__name__ == "__main__"):
    #Dados de entrada
    LstDisciplinas = [
        "Matemática (Profa. Sandra)",
        "Física (Prof. Costa)",
        "Química (Prof. Lima)"
    ]

    LstHorarios = ["Seg-8h", "Seg-10h", "Ter-8h", "Ter-10h", "Qua-8h"]

    LstRestricoes = [
        ("Prof. Silva", "Ter-8h"),
        ("Prof. Lima", "Seg-8h"),
        ("Prof. Lima", "Seg-10h")
    ]

    #Gera horários
    solucao = GerarHorarios(LstDisciplinas, LstHorarios, LstRestricoes)
    if(solucao):
        print("Grade de Horários Gerada:")
        for disciplina, horario in solucao:
            print(f"{disciplina}: {horario}")
    else:
        print("Não foi possível gerar uma grade sem conflitos!")
#Fim do programa "ControleDeHorarios" -----

```